Data Structures: Core Concepts

A quick-reference guide to foundational data organization layouts and complexities

1. INTRODUCTION

A data structure is a specialized format for organizing, processing, storing, and retrieving data inside memory. Choosing the right structure balances operational execution speeds with resource usage.

2. LINEAR DATA STRUCTURES

Data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements.

I. Arrays & Lists

Fixed or dynamic contiguous blocks of memory containing elements indexed from `0`.

- **Lookup:** `O(1)` if the target index integer is known.
- **Insertion/Deletion:** `O(n)` as shifting neighboring elements is required.

II. Linked Lists

Elements (nodes) are distributed non-contiguously in memory, where each node points to the next node using memory addresses.

- **Pros:** Dynamic sizing; rapid insertions and structural append manipulations.
- **Cons:** No direct random access; sequentially searching requires linear tracking.

III. Stacks & Queues

Restricted access linear models for managing order of operation flows:

- **Stack (LIFO - Last In, First Out):** Operations occur at one end (e.g., Undo histories). Uses `push()` and `pop()`.
- **Queue (FIFO - First In, First Out):** Items enter at back (rear) and exit at front (e.g., printing jobs).

3. NON-LINEAR DATA STRUCTURES

I. Trees (Hierarchical)

Nodes connected by edges in a parent-child structure. The topmost element is the **Root**, and terminating objects are called **Leaves**.

- **Binary Search Tree (BST):** Left children are smaller than parent node; right children are larger. Enables fast `O(log n)` search operations under balanced configurations.

II. Graphs (Networked)

A collection of nodes (vertices) connected non-linearly by paths (edges). Used to model map networks, social connections, and system routing pathways.

III. Hash Tables / Maps

Maps distinct custom unique keywords to data records using a mathematical hashing function algorithm. Delivers blazing fast near-instant constant-time `O(1)` insertions, updates, and target lookup capabilities.